

JavaScript Avanzado (2.ª parte)

10 ejercicios prácticos — prototipos, memoria, npm, transpiladores y RxJS

DOCUMENTO DE REFERENCIA

Frontend 02

Estos ejercicios repasan la herencia prototipal, las funciones constructoras y clases, la gestión de memoria, npm, Babel y los conceptos fundamentales de RxJS.

Fácil (1-3)

Fácil-Medio (4-7)

Medio (8-10)

1 Herencia basada en prototipos

FÁCIL

Explica con tus palabras la diferencia entre la herencia basada en clases (de otros lenguajes) y la herencia basada en prototipos de JavaScript.

2 hasOwnProperty

FÁCIL

Explica qué hace `Object.prototype.hasOwnProperty(prop)` y pon un ejemplo de código donde se use sobre un objeto `coche` para comprobar si tiene la propiedad `marca` como propiedad propia.

3 Función constructora

FÁCIL

Crea una función constructora `Persona(nombre, edad)` que asigne ambos parámetros como propiedades del objeto. Después crea dos instancias, `personal` y `persona2`, con datos distintos.

4 De función constructora a class

FÁCIL-MEDIO

Reescribe la función constructora `Persona` del ejercicio anterior como una `class` de ES6, añadiendo un método `saludar()` que muestre "Hola, soy [nombre]".

5 Herencia con extends y super

FÁCIL-MEDIO

Crea una clase `Estudiante` que extienda de `Persona` mediante `extends`, añadiendo una propiedad `curso`. Usa `super()` en el constructor para inicializar `nombre` y `edad`.

6 Causas de memory leaks

MEDIO

Enumera tres causas comunes de memory leaks mencionadas en el documento y, para cada una, explica brevemente cómo se podría evitar.

7 npm y package.json

MEDIO

Explica para qué sirve el archivo `package.json` en un proyecto Node.js y escribe el comando npm necesario para instalar la librería `lodash` como dependencia del proyecto.

8 Transpiladores y Babel

MEDIO

Explica qué hace un transpilador como Babel y por qué es necesario instalar el paquete `@babel/preset-env` al configurarlo.

9 RxJS: observable y observer

MEDIO

Escribe el código de un observable simple llamado `numeros$` que emita los valores 1, 2 y 3, y un observer que los reciba en la consola mediante su callback `next`. Suscribe el observer al observable.

10 RxJS: operador map y Subject

MEDIO

A partir del observable `numeros$` del ejercicio anterior, usa el operador `map` para crear un nuevo observable que emita cada valor multiplicado por 2. Después, explica la diferencia entre un `Observable` normal y un `Subject` en cuanto a unicast/multicast.

Soluciones orientativas

1. En la herencia basada en clases, los objetos heredan de una plantilla de clase; en JavaScript, los objetos heredan directamente de otros objetos a través de la cadena de prototipos (`[[Prototype]]`).
2. Devuelve `true` si la propiedad existe directamente en el objeto (no heredada): `coche.hasOwnProperty('marca')` .
3.

```
function Persona(nombre, edad){ this.nombre = nombre; this.edad = edad; } const personal = new Persona('Ana', 30); const persona2 = new Persona('Luis', 25);
```
4.

```
class Persona { constructor(nombre, edad){ this.nombre = nombre; this.edad = edad; } saludar(){ console.log(`Hola, soy ${this.nombre}`); } }
```
5.

```
class Estudiante extends Persona { constructor(nombre, edad, curso){ super(nombre, edad); this.curso = curso; } }
```
6. Variables globales innecesarias (evitarlas o liberarlas); event listeners no eliminados cuando ya no se usan (usar `removeEventListener`); temporizadores no limpiados (`clearTimeout` / `clearInterval` cuando ya no sean necesarios).
7. `package.json` define nombre, versión, dependencias y scripts del proyecto. Para instalar lodash: `npm install lodash` (se añade a `dependencies`).
8. Babel convierte código JavaScript moderno (ES6+) a una versión compatible con navegadores más antiguos (como ES5). `@babel/preset-env` es el preset que decide automáticamente qué transformaciones aplicar según los navegadores objetivo.
9.

```
const numeros$ = new Observable(subscriber => { subscriber.next(1); subscriber.next(2); subscriber.next(3); subscriber.complete(); }); const observer = { next: v => console.log(v) }; numeros$.subscribe(observer);
```
10. `const doble$ = numeros$.pipe(map(v => v * 2));` . Un Observable normal es unicast (cada suscriptor dispara su propia ejecución independiente); un Subject es multicast (todos los suscriptores comparten la misma ejecución y reciben los mismos valores).

Practica primero sin mirar las soluciones — es la mejor forma de consolidar lo aprendido.